

AI Personal Finance Coach

Complete Project Development Prompt — Documentation

1. Project Overview

Build a production-grade AI-powered Personal Finance Coach that helps users understand, analyze, and improve their finances while maintaining maximum privacy and security.

The application should provide intelligent insights, budgeting assistance, spending analytics, financial health scoring, forecasting, and AI-powered recommendations.

Look and feel: a premium, Apple-designed SaaS product — modern, elegant, highly animated, minimal, and extremely responsive.

Deployment target: Vercel (Frontend) and Railway / Render (Backend) with PostgreSQL. Entire codebase must be production-ready and follow clean architecture.

2. Problem Statement

Most budgeting apps only record expenses and display charts. Users still don't understand where their money goes, why they overspend, how to save more, whether they're financially healthy, or how to reach goals faster.

Users are also extremely concerned about privacy when uploading financial documents. Existing apps often store uploaded statements permanently, require bank access, store sensitive banking info, or use financial data for AI training.

This project's solution: secure AI analysis without permanently storing sensitive documents or exposing personal banking information.

3. Project Goals

Create an intelligent finance assistant that:

- Analyzes spending
- Creates budgets
- Generates AI insights
- Forecasts future expenses
- Tracks savings goals
- Detects subscriptions
- Explains spending habits

...while maintaining:

- Maximum privacy
- Zero data leakage
- Secure authentication
- Secure storage
- AI privacy guarantees

Target Audience

- Students
- Working professionals
- Freelancers
- Families
- Small business owners

4. Core Features

4.1 Authentication

Complete authentication system including:

- Email/password login
- Google Sign-In
- Apple Sign-In
- Password reset
- Email verification
- Session management
- Remember Me
- Device management
- Multi-device support
- Logout from all devices

Passwords must never be stored in plaintext — use **Argon2** hashing.

4.2 User Dashboard

Analytics dashboard showing: Total Income, Total Expenses, Current Savings, Monthly Budget, Spending Categories, Financial Health Score, Upcoming Bills, and AI Suggestions.

4.3 Expense Management

- Add / edit / delete expenses
- Categorize expenses
- Upload receipts
- Upload bank statements
- Import CSV, Excel, or PDF statements

4.4 AI Categorization

Automatically categorize transactions into: Food, Rent, Shopping, Healthcare, Education, Transportation, Entertainment, Utilities, Investments, Insurance, Others.

4.5 AI Financial Coach

Users can ask natural-language questions such as “How can I save ■5000 this month?” or “What subscriptions can I cancel?” The AI answers using only the user's financial summaries — never raw banking information.

4.6 Financial Health Score

Score generated from: Savings rate, Budget adherence, Emergency fund, Subscription load, Spending diversity, Income stability.

4.7 Budget Planner

Users create budgets with progress bars, and receive notifications before budgets are exceeded.

4.8 Goal Tracker

Users create goals (New Laptop, Vacation, Emergency Fund, Car, House) and see progress, remaining amount, and estimated completion date.

4.9 Forecasting

Predict next month's expenses, savings, and budget overruns using historical spending data.

4.10 Reports

Generate Weekly, Monthly, and Annual reports. Export as PDF or CSV.

5. AI Requirements

The AI is an assistant — **not** a financial advisor. A disclaimer must always be displayed.

The AI should: explain finances simply, generate spending insights, summarize transactions, suggest savings, forecast spending, and answer finance questions.

6. AI Privacy Rules (Critical)

This application must never compromise user privacy.

6.1 Uploaded Documents

After OCR, parsing, and AI analysis, the uploaded document must be permanently deleted from the server — no copies, no backups, no cache, no logs, no temporary folders. Deletion must be automatic.

6.2 AI Restrictions

The AI must never store: bank account numbers, IFSC codes, debit/credit card numbers, CVV, PIN, passwords, OTP, Aadhaar, PAN, UPI IDs, or cheque numbers.

The AI must immediately redact these values before analysis, e.g.:

- Account Number → XXXXXXXX4521
- Card Number → XXXX XXXX XXXX 5621

The AI should only retain masked versions temporarily in memory during processing. Nothing is persisted.

6.3 AI Memory

Conversation memory is disabled. Every chat only accesses the user's structured financial summaries stored in the database — never the original uploaded documents. User financial information is never used for model training.

7. Security Requirements

This is a finance application — security is the highest priority.

7.1 Authentication Security

- JWT access tokens
- Refresh tokens
- HttpOnly cookies
- Secure cookies
- SameSite Strict
- Session expiration
- Device tracking

7.2 Password Security

- Argon2 hashing
- Minimum 12 characters
- Password strength meter
- Rate limiting
- Brute force protection

7.3 API Security

- CSRF protection
- CORS
- Input validation
- SQL injection prevention
- XSS protection
- Helmet
- CSP headers
- HSTS
- Request validation
- API rate limiting

7.4 Database

Encrypt sensitive fields. Use parameterized queries only. Never expose database IDs publicly — use UUIDs.

7.5 Logging

Never log: account numbers, uploaded statements, passwords, OTPs, or personal financial details.
Only log technical errors.

7.6 File Upload Security

Allow only PDF, CSV, XLS, XLSX. Maximum size: 10MB. Virus scan uploads. Reject executable files.
Delete uploads immediately after processing.

7.7 Encryption

Encrypt sensitive user data, financial summaries, and refresh tokens. TLS everywhere.

7.8 User Controls

- Delete account
- Export all data
- Delete all uploaded data
- Delete AI chat history
- Delete reports

8. UI / UX Design

Create an ultra-premium experience inspired by modern Apple design language.

8.1 Theme

Dark mode by default, with a beautiful light mode. Theme switching should animate smoothly.

8.2 Design Style

- Glassmorphism
- Frosted glass cards
- Soft blur backgrounds
- Rounded corners
- Floating panels
- Thin borders
- Premium typography
- Spacious layouts
- Elegant shadows
- Minimal icons

8.3 Color Palette

Dark: Black, Graphite, Deep Blue, Emerald, White text. **Accent:** Cyan, Emerald, Soft Purple. No bright colors.

8.4 Animations

Should feel alive without being distracting (60 FPS, never compromising usability):

- Smooth page transitions
- Scroll-reveal sections
- Cursor-responsive floating cards
- Subtle ripple/glow buttons
- Fade & slide text
- Parallax background movement
- Animated gradients
- Fluid loading animations

- Seamless dashboard transitions
- Smooth chart animations
- Premium hover interactions
- Spring-based micro-interactions
- Animated counters
- Elegant onboarding sequence
- Skeleton loading placeholders
- Animated navigation indicator

8.5 Landing Page

Includes: Hero section (headline, interactive illustration, animated dashboard preview), Features, Security, Privacy, Testimonials, Pricing, FAQ, Footer.

8.6 Dashboard Pages

- Dashboard
- Expenses
- Income
- Budgets
- Goals
- Reports
- AI Coach
- Settings
- Profile
- Security Center

8.7 Settings

- Theme
- Language
- Notifications
- Privacy
- Connected Devices
- Delete Account
- Download Data
- Manage Sessions

9. Tech Stack

9.1 Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS
- Framer Motion

- GSAP (where appropriate)
- React Query
- Recharts

9.2 Backend

- FastAPI
- Python

9.3 Database

- PostgreSQL

9.4 Authentication

- JWT
- OAuth
- Argon2

9.5 Storage

Temporary encrypted object storage only for document processing, with automatic deletion immediately after analysis.

9.6 AI

- OpenAI or Gemini
- LangChain (optional)
- OCR
- Document parser

9.7 Deployment

- Frontend: Vercel
- Backend: Railway / Render
- Database: Supabase PostgreSQL

10. Code Quality Standards

- Modular architecture
- Clean folder structure
- SOLID principles
- Type safety
- Unit tests
- Integration tests
- API documentation
- Docker support
- CI/CD pipeline

- Environment variables
- Comprehensive README
- Production-ready configuration

11. Final Goal

The finished application should feel like a premium SaaS product that could realistically compete with modern personal finance platforms. Every interaction should emphasize clarity, elegance, privacy, and trust.

Users should immediately understand that their financial data is treated with the highest level of security: uploaded documents are processed in-memory or in encrypted temporary storage and automatically deleted after analysis, sensitive identifiers are redacted before reaching the AI model, and no banking credentials or personal financial information are retained or used for model training.

The experience should be polished, fast, accessible, and secure enough that users feel confident trusting it with their financial analysis, while retaining complete control over their data.